

Softwarekonstruktion WS 15/16 – Übung 5

Ausgabe: 04.01.2016, Abgabe: 18.01.2016 12:00 Uhr

Präsenzübung

5.1 White-Box-Testen

5.1.1 Kontrollflussbezogenes Testen – Testfälle

Das gegebene Programmsegment soll mit einem Kontrollfluss-Testverfahren getestet werden:

```
1 read (x,y,z)
2 if ( (x>0) AND (y>0) AND (z>0) ) { x:=x*y*z }
3 else { x:=y }
4 if ( x+y+z>0 ) { z:=-x }
```

1. Geben Sie eine minimale Testmenge für eine Anweisungsüberdeckung an.

Lösungsvorschlag

$\{(1,1,1);(-1,-1,-1)\}$

2. Geben Sie eine minimale Testmenge für eine einfache Bedingungsabdeckung an.

Lösungsvorschlag

$\{(1,1,-1);(-1,-1,1)\}$

3. Geben Sie eine minimale Testmenge für eine Mehrfachbedingungsabdeckung an.

Lösungsvorschlag

$\{(1,1,1);(1,1,-1);(1,-1,1);(1,-1,-1);(-1,1,1);(-1,1,-1);(-1,-1,1);(-1,-1,-1)\}$

4. Geben Sie eine minimale Testmenge für eine minimal bestimmende Mehrfachbedingungsabdeckung an.

Lösungsvorschlag

$\{(1,1,1);(1,1,-1);(1,-3,1);(-1,1,1)\}$

5. Geben Sie eine minimale Testmenge für eine Pfadabdeckung an.

Lösungsvorschlag

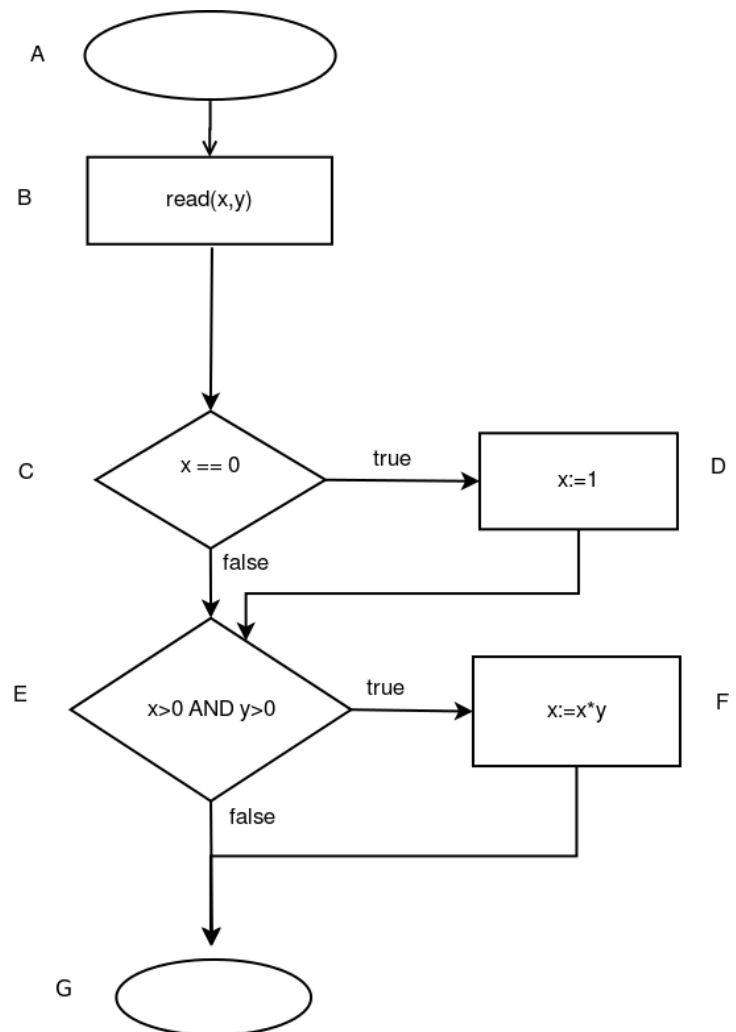
Es gibt keine Pfadabdeckung, die die Bedingungen “ $(x > 0) \text{ AND } (y > 0) \text{ AND } (z > 0)$ ” und “ $(x+y+z > 0)$ ” voneinander abhängig sind und deswegen $x + y + z > 0$ immer gilt, wenn $((x > 0) \text{ AND } (y > 0) \text{ AND } (z > 0))$ gilt.

5.1.2 Kontrollflussbezogenes Testen – Testerfüllung

Das gegebene Programmsegment soll getestet werden:

```
1 read(x,y)
2 if ( x==0 ) { x:= 1 }
3 if ( (x>0) AND (y>0) ) { x:=x*y }
```

Nehmen Sie für die Aufgaben 3-7 an, es würde ein Test mit den Testdaten $(x=0, y=1)$ und $(x=-1, y=5)$ durchgeführt.



1. Skizzieren Sie das Flussdiagramm.
2. Geben Sie alle Entscheidungswege an.

Lösungsvorschlag

$\{A, B, C\}; \{C, D, E\}; \{C, E\}; \{E, G\}; \{E, F, G\}$

3. Erfüllt diese Testmenge die Anweisungsüberdeckung? Begründen Sie Ihre Antwort.

Lösungsvorschlag

Dies Testmenge erfüllt die Anweisungsüberdeckung. Mit dem Test $(x=0, y=1)$ werden alle Anweisungen durchlaufen.

4. Erfüllt diese Testmenge die Zweigüberdeckung? Begründen Sie Ihre Antwort.

Lösungsvorschlag

Die Testmenge erfüllt die Zweigüberdeckung. Der erste Test durchläuft die jeweiligen 'true'-Zweige, der zweite Test durchläuft die entsprechenden 'false'-Zweige, so dass alle Zweige durchlaufen werden.

5. Erfüllt diese Testmenge die Entscheidungsüberdeckung? Begründen Sie Ihre Antwort.

Lösungsvorschlag

Die Testmenge erfüllt die Entscheidungsüberdeckung. Der erste Test durchläuft die jeweiligen 'true'-Zweige, der zweite Test durchläuft die entsprechenden 'false'-Zweige, so dass alle Entscheidungen einmal 'true' und einmal 'false' sind.

6. Erläutern Sie den Unterschied zwischen einer Entscheidungs- und einer Zweigüberdeckung. Was ändert sich, wenn man den Abbruch von Tests mit einbezieht?

Lösungsvorschlag

Bei Betrachtung vollständiger Testpfade kommen beide zu demselben Ergebnis. Betrachtet man bei unvollständigen Testpfaden den Überdeckungsgrad, so haben beide eine unterschiedliche Metrik.

7. Erfüllt diese Testmenge die einfache Bedingungsüberdeckung? Begründen Sie Ihre Antwort.

Lösungsvorschlag

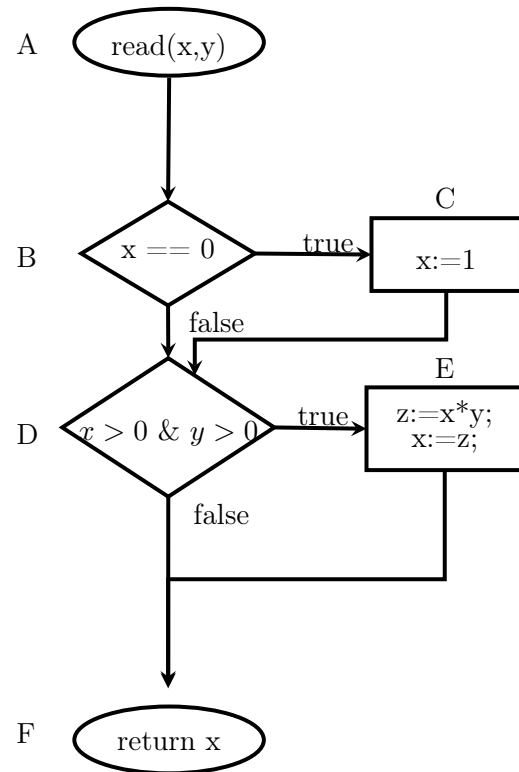
Die Testmenge erfüllt die einfache Bedingungsüberdeckung nicht, da das atomare Prädikat $y > 0$ nie zu false ausgewertet wird.

5.1.3 Datenflussbezogenes Testen

Gegeben sei das Programmsegment:

```
1. read(x,y)
2. if ( x==0 ) { x:= 1 }
3. if ( (x>0) AND (y>0) )
   { z:=x*y; x:=z;}
4. return x;
```

und das zugehörige Flussdiagramm:



1. Geben Sie für jeden der Knoten k im Kontrollflußdiagramm die Mengen $DEF(k)$, $UNDEF(k)$ und $REF(k)$ an.

Lösungsvorschlag

	A	B	C	D	E	F
$DEF(k)$	{x,y}	{ }	{x}	{ }	{z,x}	{ }
$UNDEF(k)$	{ }	{ }	{ }	{ }	{ }	{ }
$REF(k)$	{ }	{x}	{ }	{x,y}	{x,y,z}	{x}

2. Was fällt bei Knoten E auf? Welche Konsequenz ergibt sich daraus?

Lösungsvorschlag

Es liegt ein rein lokaler Datenfluss vor: z wird erst definiert und dann referenziert. Der Knoten muss deswegen in zwei Knoten aufgeteilt werden: $E_1 : z := x*y$ und $E_2 : x := z$. Daraus ergeben sich dann $DEF(E_1)=\{z\}$, $REF(E_1)=\{x,y\}$ und $DEF(E_2)=\{x\}$, $REF(E_2)=\{z\}$.

3. Gegeben seien nun die Testdaten: $\{x=0, y=1\}$. Bestimmen Sie welche der folgenden Kriterien von Datenflüssen erfüllt sind und begründen Sie Ihre Aussage.

Alle Definitionen

Alle DR-Interaktionen

Alle Referenzen

Lösungsvorschlag

Alle Definitionen Ja, alle Zuweisungen werden auch mindestens einmal verwendet.

Alle DR-Interaktionen Nein, vom Knoten A wird mit den Testdaten Knoten D nicht erreicht. Da dieser aber über den Knoten B erreichbar wäre, erfüllen die Testdaten die Voraussetzungen für alle DR-Interaktionen nicht.

Alle Referenzen Nein, es werden nicht jeweils alle Zweige nach den Entscheidungsknoten durchlaufen. z.B. nicht $B \rightarrow D$.

Hausübung

5.2 White-Box-Testen (20 Punkte)

Hinweis zu dieser Hausübung: Diese Aufgabe muss von jedem Teilnehmer in Moodle durchgeführt und abgeschlossen werden, kann jedoch in Gruppen besprochen werden. Die Bearbeitung *dieser* Aufgabe ist möglich bis zum 19. Januar 2016 4:00 Uhr.

Die übrigen Aufgaben müssen auf Papier in den Briefkasten 21, OH 12 eingeworfen werden (Gruppenabgabe: bis zu vier Personen einer Übungsgruppe).

5.3 Konfigurationsmanagement (20 Punkte)

5.3.1 Versionskontrolle (6 Punkte)

Beschreiben Sie das *Pessimistische* Modell der Versionskontrolle:

1. Beschreiben Sie die einzelnen Schritte, wenn 2 Personen eine Datei bearbeiten wollen.
2. Welche Probleme können bei dem *Pessimistischem* Modell im Allgemeinen auftreten?

1. Beschreiben Sie die einzelnen Schritte, wenn 2 Personen eine Datei bearbeiten wollen:
 - a) Person A sperrt Datei
 - b) Person A bearbeitet Datei
 - c) Person A gibt Datei frei
 - d) Person B sperrt Datei
 - e) Person B bearbeitet Datei
 - f) Person B entsperrt Datei
2. Welche Probleme können bei dem *Pessimistischem* Modell im Allgemeinen auftreten?
 - a) Entsperren kann vergessen werden.
 - b) Paralleles Arbeiten an einer Datei nicht möglich
 - c) Deadlock ist möglich.

5.3.2 GIT (8 Punkte)

Gegeben seien die 4 aufeinander folgenden Commits *A, B, C, D* auf dem Branch *master*. Commit *A* enthält drei Dateien *main.c* *menu.c* und *menu.h*, von denen *menu.c* und *menu.h* im Commit *B* gelöscht wurden. Commit *C* ändert *main.c*. Vom Branch *master* soll ein Branch *branch0* abgespalten werden, der alle Änderungen bis inklusive *C* und zusätzlich die in *B* gelöschten Dateien enthält. Angenommen, Sie befinden sich aktuell auf dem Branch *master* nach Commit *D*:

1. Geben Sie die notwendigen Befehle an, um den beschriebenen Branch *branch0* zu erstellen. Wählen Sie eine möglichst kurze Variante. Gehen Sie dabei von aus, dass in Commit

B nur die Dateien gelöscht wurden. Sollten Sie Commit IDs benötigen, gehen Sie davon aus, dass diese gleich der Bezeichnungen der Commits sind.

Lösungsvorschlag

```
git branch branch0 C
git checkout branch0
git revert B
```

2. Nachdem der Branch *branch0* wie oben beschrieben erstellt wurde, wird ein Commit E hinzugefügt. Gleichzeitig wird auf dem Branch *master* ein Commit F hinzugefügt. Nun soll der Branch *branch0* so in den Master gemergt werden, dass alle Änderungen auf dem Branch in Master enthalten sind, bis auf die in Commit B gelöschten Dateien. Geben Sie auch hier die Notwendigen git Befehle an.

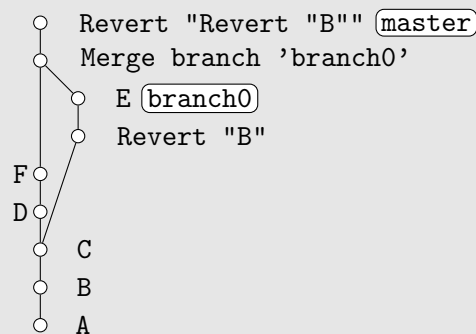
Lösungsvorschlag

Folgende Befehle reichen aus:

```
git checkout master
git merge branch0
git revert branch0~1
```

3. Zeichnen Sie den GIT-Graphen zu den dazugehörigen Commits aus Aufgabe 1 und 2.

Lösungsvorschlag



Schauen Sie auch bei Bedarf auch in das Buch *Pro GIT* <https://git-scm.com/book/en/v2>

5.3.3 Releasemanagement (6 Punkte)

1. Nennen Sie die einzelnen Stages der Deployment Pipeline und erläutern Sie diese.

Lösungsvorschlag

- **Commit Stage:** Code wird kompiliert.
 - Durchführung von Unit Tests und Code Analyse.
 - Packen der Software in ein Format (jar, war, PE).
- **Acceptance Test Stage:** Test von User Stories.
 - Im Gegensatz zu Unit Tests werden hier Anwendungsfälle bzw. funktionale Eigenschaften getestet.
- **Manual Test Stage:**
 - Explorative Tests
- **Performance Test Stage:**
 - Testen von nicht funktionalen Eigenschaften, wie Performanz, Skalierbarkeit, Durchsatz, Verfügbarkeit usw
- **Release Stage:**
 - Software auf Produktivsystem installiert oder als installierbare Software gepackt

2. Beschreiben Sie die beiden Deployments *Blue-Green Deployment* und *Kanarienvogel Release*.

- a) Wie werden diese durchgeführt?
- b) Besitzen Diese Gemeinsamkeiten? Wenn ja welche?

Lösungsvorschlag

Blue-Green Deployment Beim Blue-Green Deployment werden für ein System jeweils ein Blue-Slice und ein Green-Slice vorrätig gehalten, die jeweils verschiedene Versionen des selben Systems darstellen. Zu jeder Zeit ist eine Version aktiv, während die andere weiterentwickelt wird. Ist eine neue Version bereit, kann auf diese umgeschaltet werden. Treten Fehler auf, kann wieder die alte Version aktiv geschaltet werden.

Kanarienvogel Release Beim Kanarienvogel Release wird eine Neue Version einer Software zunächst an eine kleine Teilmenge der Zielsysteme verteilt, während auf der größeren Teilmenge die alte Version verbleibt. Bei Problemen kann wieder die alte Version verwendet werden. Treten keine Probleme auf, kann die Teilmenge vergrößert werden,

Gemeinsamkeiten Sowohl beim Blue-Green Deployment beim Kanarienvogel Release, werden zwei Versionen vorrätig gehalten.